

N100 Financial Intelligence Platform

Daily Reports

Project progress record, 45-day internship build

9 day-level logs | 6 sprint retrospectives

Compiled 2026-07-28
Raj Sarania, Data Analyst
Summer Internship Programme, Bluestock Fintech

Coverage map

What level of record exists for each part of the project.

Read this first

This project was recorded at two levels of granularity. Day-level log files exist for 9 days: 4, 5, 6, 7, 8, 9, 10, 11, 12. The remaining days were recorded as stand-ups submitted through the internship portal and consolidated into the six sprint retrospectives reproduced in Part 2 of this document. No day-level entry has been reconstructed after the fact: where a daily log does not exist, this document says so rather than presenting a sprint record as a daily one.

Sprint	Days	Focus	Record level
Sprint 1	Days 1-7	Data Foundation	Day-level logs for Days 4 to 7
Sprint 2	Days 8-14	Financial Ratio Engine	Day-level logs for Days 8 to 12
Sprint 3	Days 15-21	Screeener & Peer Comparison	Sprint-level retrospective
Sprint 4	Days 22-28	Dashboard & Valuation	Sprint-level retrospective
Sprint 5	Days 29-35	Intelligence, NLP & PDF Reports	Sprint-level retrospective
Sprint 6	Days 36-45	API Server, Clustering & Final QA	Sprint-level retrospective

Document structure

Part 1 reproduces every day-level project log, in day order, as written at the time.

Part 2 reproduces the six sprint retrospectives, which carry the day-by-day detail for the remainder of the project including the decisions taken, defects found and technical debt recorded.

Part 1 - Day-level project logs

Reproduced as written at the time, in day order.

Day 4 project log

Source: *Day4_Project_Log_N100.docx*

Bluestock SIP - N100 Financial Intelligence Platform

Day 4 Project Log

Purpose: Preserve complete Day 4 context for future ChatGPT sessions and project continuity.

Objectives Completed

Implemented SQLite database loader.

Created schema.sql.

Generated data/nifty100.db.

Loaded all 7 core datasets.

Verified all row counts against Excel sources.

Repository Changes

SQLite implementation moved to src/etl/sqlite_loader.py.

Schema moved to src/etl/schema.sql.

Removed separate src/db architecture to align with project documentation.

Database standardized as data/nifty100.db.

Verification

Tables: analysis, balancesheet, cashflow, companies, documents, profitandloss, prosandcons.

Verified row counts: 20, 1312, 1187, 92, 1585, 1276, 16 respectively.

Total rows inserted: 5488.

Decisions

N100_Project_Document has highest priority.

Sprint Guide has second priority.

README is third priority.

Engineering best practices only when documentation is silent.

Issues & Resolutions

Resolved ModuleNotFoundError by restructuring and executing module correctly.

Fixed coding typos encountered during implementation.

Confirmed SQLite loader works end-to-end.

Current Sprint Status

Day 1 Complete

Day 2 Complete

Day 3 Complete

Day 4 Complete

Next Session (Day 5)

Load 5 supporting datasets.

Create remaining SQLite tables.

Load all 12 datasets.

Generate load_audit.csv.

Verify all table counts.

Workflow

Requirement → Implement → Verify → Commit.

Always check project documents before coding.

Day 5 project log

Source: *Day5_Project_Log_N100.docx*

Day 5 Project Log - N100 Financial Intelligence Platform

Purpose: Preserve complete Day 5 context for future ChatGPT sessions and project continuity.

Objectives Completed

Extended schema.sql with five supplementary tables.

Updated sqlite_loader.py to load all 12 datasets.

Added support for supporting datasets using header=0.

Generated reports/load_audit.csv.

Rebuilt data/nifty100.db and loaded all datasets successfully.

Verification

Dataset

Rows

Status

analysis

20

PASS

balancesheet

1312

PASS

cashflow

1187

PASS

companies

92

PASS

documents

1585

PASS

profitandloss

1276

PASS

prosandcons

16

PASS

financial_ratios

1184

PASS

market_cap

552

PASS

peer_groups

56

PASS

sectors

92

PASS

stock_prices

5520

PASS

Total rows loaded: 12,892

Audit Summary

load_audit.csv generated successfully.

rows_in equals rows_out for every table.

Rejected records: 0 for all datasets.

All 12 SQLite tables verified.

Repository Changes

src/etl/schema.sql

src/etl/sqlite_loader.py

reports/load_audit.csv

data/nifty100.db

Important Decisions

Project priority followed throughout: N100_Project_Document > Sprint Guide > Repository > Engineering Best Practices.

Supporting datasets use header=0; core datasets continue to use header=1.

Loader was extended with minimal changes to preserve Day 4 implementation.

Sprint Status

Day 1 Complete

Day 2 Complete

Day 3 Complete

Day 4 Complete

Day 5 Complete

Next Session (Day 6)

Perform manual data quality review for five random companies.

Check consistency across time-series tables.

Fix loader issues only if discovered.

Document DQ review findings.

Day 6 project log

Source: *Day6_Project_Log_N100.docx*

Day 6 Project Log - N100 Financial Intelligence Platform

Purpose: Preserve Day 6 context for future ChatGPT sessions and project continuity.

Objectives Completed

Created `src/etl/dq_review.py` for post-load SQLite verification.

Verified SQLite connectivity, table existence, and row counts for all 12 datasets.

Used `tempo.ipynb` as the investigation sandbox before modifying production code.

Performed manual review of five randomly selected companies.

Verified cross-table relationship: `companies.id` is the business key referenced by `company_id` in financial datasets.

Investigated reporting-period consistency across Balance Sheet, Cash Flow, and Profit & Loss tables.

Confirmed duplicate records are inherited from the source Excel datasets rather than introduced by the ETL pipeline.

Prepared `reports/dq_review_report.md` documenting the investigation and conclusions.

Key Findings

All 12 SQLite tables matched expected row counts.

`companies.id` stores ticker/business identifiers referenced by `company_id` in other datasets.

`analysis` and `prosandcons` are intentionally sparse source datasets.

Time-series differences (quarterly, annual, TTM) reflect source reporting practices.

Duplicate records observed for ASIANPAINT and BAJAJ-AUTO match Day 3 validation and are source-data characteristics.

Engineering Decisions

Follow project priority: N100_Project_Document > Sprint Guide > Repository > Engineering best practices.

Use `tempo.ipynb` for experiments; only validated logic reaches production code.

`dq_review.py` is an objective verification utility.

`dq_review_report.md` contains investigation findings and conclusions.

Repository Changes

`src/etl/dq_review.py`

`reports/dq_review_report.md`

Sprint Status

Day 1 Complete

Day 2 Complete

Day 3 Complete

Day 4 Complete

Day 5 Complete

Day 6 Complete

Next Session

Audit Sprint 1 deliverables against project documentation.

Address any remaining Sprint 1 gaps.

Prepare to begin Sprint 2 (Financial Ratio Engine).

Day 7 project log

Source: *Day7_Project_Log_N100.docx*

Day 7 Project Log – Sprint 1 Completion

Objective

Complete Sprint 1 deliverables by validating the SQLite warehouse, finalizing exploratory SQL, reviewing repository readiness, and preparing project documentation.

Work Completed

Audited Sprint 1 deliverables against the updated Sprint 1 guide.

Reviewed repository structure and validated core ETL, validation, database and reports.

Finalized `exploratory_queries.sql` with 11 validated exploratory SQL queries.

Verified company coverage, NULL checks, reporting period coverage and duplicate company-year detection.

Reviewed the company ID primary-key decision; deferred structural refactoring until justified by future requirements.

Completed Sprint 1 retrospective.

Confirmed all automated tests pass (44 passed).

Key Findings

Eight company IDs exist in financial datasets but are absent from the companies master table.

The Analysis dataset intentionally contains partial company coverage.

Duplicate company-year records were detected for ADANIPORTS in Profit & Loss.

Reporting periods include annual values and TTM entries.

Sprint Status

Sprint 1 technical work is complete. Remaining activities are repository administration, Git commit, and push before beginning Sprint 2.

Day 8 project log

Source: *Day8_Project_Log_N100.docx*

Day 8 Project Log - N100 Financial Intelligence Platform

Purpose: Preserve Day 8 context for future ChatGPT sessions and project continuity.

Objectives Completed

Created `src/analytics/ratios.py`.

Implemented Net Profit Margin (NPM).

Implemented Operating Profit Margin (OPM) with source cross-validation.

Implemented Return on Equity (ROE).

Implemented Return on Capital Employed (ROCE).

Added edge-case handling and logging.

Validated calculations using `tempo.ipynb`.

Reviewed and cleaned code before Day 9.

Verification

NPM verified (20.0, -10.0, None).

OPM verified including mismatch logging and zero-sales handling.

ROE verified for positive, negative and zero-equity cases.

ROCE verified (64.0%, 100.0%, None).

Engineering Decisions

Sprint Guide is the primary implementation reference.

N100 Project Document is used as the business logic reference.

`tempo.ipynb` remains the sandbox for experimentation.

Repository Changes

`src/analytics/ratios.py`

Sprint Status

Sprint 2 Day 8 Complete.

Next Session

Implement Debt-to-Equity.

Implement Interest Coverage Ratio.

Implement Asset Turnover.

Begin KPI tests under `tests/kpi/`.

Day 9 project log

Source: *Day9_Project_Log_N100.docx*

Day 9 Project Log - N100 Financial Intelligence Platform

Purpose: Preserve Day 9 context for future ChatGPT sessions and project continuity.

Objectives Completed

Implemented Debt-to-Equity ratio.

Implemented High Leverage flag with Financials sector exemption.

Implemented Interest Coverage Ratio (ICR).

Implemented ICR display label ('Debt Free').

Implemented ICR warning flag.

Implemented Net Debt calculation.

Implemented Asset Turnover ratio.

Created tests/kpi/test_ratios.py covering Day 8 and Day 9 ratio functions.

Verified logic using tempo.ipynb before production updates.

Executed full pytest suite successfully.

Verification

Debt-to-Equity verified for normal, debt-free and invalid equity cases.

High Leverage flag verified including Financials exemption.

Interest Coverage verified including interest=0 edge case.

ICR label and warning flag verified.

Net Debt verified including negative values.

Asset Turnover verified including zero-assets edge case.

79/79 pytest tests passed successfully.

Repository Changes

src/analytics/ratios.py

tests/kpi/test_ratios.py

Engineering Decisions

Continued using tempo.ipynb as the experimentation sandbox.

Maintained single-quote coding convention.

Followed project priority: N100 Project Document > Sprint Guide > Repository > Engineering Best Practices.

Kept ratio calculations modular with separate helper functions for labels and flags.

Sprint Status

Sprint 2 Day 8 Complete

Sprint 2 Day 9 Complete

Next Session (Day 10)

Create CAGR engine (revenue, PAT and EPS).

Handle CAGR edge cases and flags.

Add CAGR unit tests.

Day 10 project log

Source: *Day10_Project_Log_N100.docx*

Day 10 Project Log - N100 Financial Intelligence Platform

Purpose: Preserve Day 10 context for future ChatGPT sessions and project continuity.

Objectives Completed

- Created `src/analytics/cagr.py`.
- Implemented reusable `calculate_cagr()` engine.
- Added Revenue CAGR, PAT CAGR and EPS CAGR wrapper functions.
- Implemented CAGR edge-case handling and flag generation.
- Validated logic in `tempo.ipynb` before production update.
- Created `tests/kpi/test_cagr.py`.
- Executed complete pytest suite successfully.

Verification

- Positive CAGR calculation verified.
- Edge cases verified: `DECLINE_TO_LOSS`, `TURNAROUND`, `BOTH_NEGATIVE`, `ZERO_BASE`.
- Invalid years handled safely.
- 91/91 pytest tests passed successfully.

Repository Changes

- `src/analytics/cagr.py`
- `tests/kpi/test_cagr.py`

Engineering Decisions

- Reused a single CAGR helper to avoid duplicated logic.
- Wrapper functions kept lightweight for future SQLite/data-pipeline integration.
- Continued using `tempo.ipynb` as the experimentation sandbox.
- Maintained single-quote coding convention.
- Followed project priority: N100 Project Document > Sprint Guide > Repository > Engineering Best Practices.

Sprint Status

- Sprint 2 Day 8 Complete
- Sprint 2 Day 9 Complete
- Sprint 2 Day 10 Complete

Next Session (Day 11)

- Begin remaining Sprint 2 KPI implementation as defined in the sprint guide.
- Integrate KPI calculations with the broader analytics pipeline as required.
- Continue expanding KPI test coverage.

Day 11 project log

Source: *Day11_Project_Log_N100.docx*

Day 11 Project Log – Sprint 2 (Cash Flow KPI Engine)

Objective

Complete the Day 11 cash-flow KPI implementation defined in the Sprint Guide and align it with the N100 Financial Intelligence Platform specification.

Work Completed

Implemented Free Cash Flow calculation.

Implemented FCF Concern Flag.

Implemented CFO Quality Score.

Implemented Average CFO Quality Score and Quality Label helper.

Implemented CapEx Intensity and CapEx Intensity Label helper.

Implemented FCF Conversion and FCF Conversion Label helper.

Completed Capital Allocation classifier with documented mappings and Unknown Pattern fallback.

Validated all functions in tempo.ipynb.

Updated KPI unit tests.

Repository now passes 109/109 pytest tests.

Key Decisions

Maintained ratios.py as the central analytics module. Separated calculation logic from label/helper logic. Implemented only documented capital allocation business rules.

Carry Forward

Review and generate output/capital_allocation.csv during Day 12 Ratio Engine integration after confirming the intended execution flow.

Repository Status

Stable • 109/109 tests passing • Ready for Sprint 2 Day 12.

Day 12 project log

Source: *Sprint2_Day12_Part1_Chat_Transfer_Log.docx*

Sprint 2 Day 12 (Part 1) - Chat Transfer Log

Purpose: Transfer only the discoveries, decisions and current implementation state from this chat so the next chat can continue Day 12 Part 2 without repeating investigation.

Initial Context

Objective: Start Sprint 2 Day 12.

Implementation authority: Sprint Guide.

Business reference: N100 Project Document.

Repository screenshot reviewed.

Rule: Use single quotes in all Python code.

tempo.ipynb reserved for debugging/experimentation.

Primary Problem Investigated

Previous blocker: Multiple dataset issue. Needed to determine the correct architecture for integrating Profit & Loss, Balance Sheet and Cash Flow into financial_ratios.

Key Discoveries

The project documentation expects one financial_ratios row per (company_id, year).

SQLite is the runtime data source for Day 12; Excel is only the ingestion source after Sprint 1.

Project documents define KPI sources but do not define merge strategy.

Repository architecture does not require restructuring.

ratio_engine.py should remain in src/etl as the orchestration layer.

Analytics modules remain calculation-only.

Schema.sql contained a missing comma after cash_from_operations_cr and was corrected.

Experimental verification using tempo.ipynb proved that INNER JOIN(profitandloss, balancesheet, cashflow) on (company_id, year) returns 1184 rows.

Verified row counts: Profit & Loss=1276, Balance Sheet=1312, Cash Flow=1187, Merged=1184.

1184 matches the supplied financial_ratios dataset, validating the merge strategy.

Duplicates are inherited from source datasets and should be preserved.

Implementation Performed

Reviewed ratio_engine.py, sqlite_loader.py, schema.sql and analytics/ratios.py.

Expanded imports in ratio_engine.py to include additional KPI modules.

Expanded KPI integration beyond NPM, OPM and ROE toward full Sprint 2 coverage.

CAGR intentionally deferred because it requires grouped multi-year processing.

Decisions Made

Do not redesign the repository.

Do not change the validated INNER JOIN strategy.

Do not use Excel directly in the Ratio Engine.

Do not deduplicate source data.

Follow Sprint Guide first and N100 Project Document second whenever implementation questions arise.

Complete KPI audit before inserting into financial_ratios.

Current Status

Merge strategy validated.

Repository review completed.

Architecture review completed.

Ratio Engine partially integrated.

financial_ratios population not yet implemented.

Resume Point for Day 12 Part 2

Step 6A: Audit all Sprint Guide required KPI columns.

Identify calculated KPIs versus pass-through source fields.

Complete missing KPI population.

Populate financial_ratios table.

Validate expected row count (~1184).

Perform manual ROE and Revenue CAGR verification.

Close Sprint 2 Day 12.

Part 2 - Sprint retrospectives

The consolidated record for the days without an individual log.

Sprint 1 retrospective

Source: docs/sprint1_retro.md

Sprint Objective

Sprint 1 focused on establishing the data foundation for the N100 Financial Intelligence Platform. The primary objectives were to ingest the provided Excel datasets, validate data quality, normalize key fields, build a SQLite warehouse, and prepare the project for downstream financial analytics.

Work Completed

- Implemented Excel data loading for all provided datasets.
- Developed normalization logic for company identifiers and reporting periods.
- Built automated data validation checks.
- Generated validation reports for failed records.
- Designed and implemented the SQLite database schema.
- Loaded all datasets into the SQLite database.
- Generated load audit reports.
- Performed manual data quality review on sampled companies.
- Created exploratory SQL queries for database validation and data exploration.
- Achieved successful execution of all unit tests (44 passed).

Key Findings

- The Companies master dataset contains 92 companies, while certain financial datasets include additional company IDs.
- The Analysis dataset intentionally provides partial company coverage.
- Reporting periods include both annual financial statements and TTM (Trailing Twelve Months) values.
- Duplicate company-year records were identified for Adani Ports in the Profit & Loss dataset and documented through exploratory SQL.

Challenges

- Understanding relationships between multiple datasets and establishing appropriate company mappings.
- Handling varying reporting period formats across companies.
- Distinguishing between expected source-data characteristics and genuine data quality issues.
- Ensuring all ETL and validation processes remained reproducible through automated testing.

Lessons Learned

- Validate source data before modifying transformation logic.
- Separate exploratory findings from implementation changes.
- Use automated validation alongside manual data quality review.
- Maintain a consistent project structure to support future sprint development.

Sprint Outcome

Sprint 1 objectives were successfully achieved. The project now has a validated SQLite database, automated ETL pipeline, data validation framework, audit reports, and exploratory SQL documentation, providing a stable foundation for Sprint 2 development.

Sprint 2 retrospective

Source: docs/Sprint2_retrospective.md

N100 Financial Intelligence Platform

Sprint: Sprint 2 - Financial Ratio Engine (Days 8-14)

Status: Completed

Prepared On: Day 14

This retrospective summarizes the decisions, technical finding, implementation outcomes, and lesson learned during Sprint 2.

This document captures why certain decisions were made, what technical debt remains, and the recommendations carried forward into Sprint 3.

Sprint 2 focused on transforming validated financial statement data into a reusable Financial Ratio Engine capable of producing standardized investment KPIs for the N100 Financial Intelligence Platform.

Primary Objectives:

- Develop reusable financial ratio calculation modules.
- Populate the `financial\_ratio` SQLite table.
- Validate calculations against sources datasets.
- Handle real world financial edge cases.
- Produce a reliable analytics foundation for later platform modules.

Ratio Engine

Successfully implemented:

Profitability

- Net profit margin
- Operating profit margin
- Return on Equity
- Return on capital employed (validation)

Leverage

- debt-to-equity
- interest coverage
- net debt

Efficiency

- Asset turnover

Cash Flow KPIs

- Free cash flow
- CFO quality score
- capital allocation classification
- CapEx intensity
- FCF conversion

CAGR Engine

Implemented reusable CAGR calculations for:

- Revenue CAGR
- PAT CAGR
- EPS CAGR

Supported business scenarios includes:

- Zero base
- company turnaround
- Decline into less
- Both values negative
- Invalid year handling

The CAGR engine had been fully unit tested and is ready for integration when schema requirements include CAGR persistence.

Operating Profit Margin

The supplied `opm\_percentage` dataset contained widespread inconsistencies.

Decision:

- Ignore dataset OPM values.
- Compute OPM using:

Operating Profit / Sales * 100

This becomes the production implementation.

Book Values Per Share

Book value calculations could not be reproduce consistently from available financial statements.

Decision:

- Use `companies.book\_value`
- Document the Limitation.
- Avoid introducing unsupported calculations.

Database Join Strategy

Final implementation:

- INNER JOIN
- Profit & Loss
- Balance Sheet
- Cash Flow
- LEFT JOIN
- companies

This preserves financial integrity while allowing incomplete company metadata.

Duplicate Records

Duplicate company-year records were investigated.

Conclusion:

Duplicate originate from the supplied dataset rather than the implementation.

Decision:

- Preserve duplicates.
- Do not silently modify source data.

Debt-to-Equity warning flags are suppressed for Banks, NBFCs and Insurance companies where high leverage is structurally normal.

ROCE calculations are retained for validation but are not currently persisted because the existing schema does not contain a ROCE field.

Sprint 2 identified several issues originating from the supplied datasets rather than the implementation.

Confirmed findings include:

- Corrupted operating profit margin values
- Missing companies
- Orphan company IDs
- Duplicate company-year records
- Missing financial fields
- Book values ambiguity
- Accounting inconsistencies

These findings are documented separately within the Sprint 2 data quality audit report.

Sprint 2 completed with:

- 109/109 automated test passing

Coverage includes:

- Ratio calculations
- validator
- ETL
- CAGR
- Financial edge cases

The following items are intentionally deferred to future sprints.

CAGR Integration

~~The CAGR analytics module has been implemented and fully tested.

Integration into the ETL pipeline has been deferred because the current `financial\_ratios` schema does not contains CARG fields.~~

****Closed on Day 15.**** This deferral was based on a mistaken premise.

`src/etl/schema.sql` already declared `revenue\_cagr\_5yr`, `pat\_cagr\_5yr`, `eps\_cagr\_5yr` and `composite\_quality\_score`; only the live database file was missing them. The columns were added and populated by `ratio\_engine.py`. See the Sprint 3 retrospective.

Configuration

Current implementation contains hardcoded configuration values such as the SQLite database path.

Future work should migrate configuration into environment variables or a dedicated configuration module.

Repository Cleanup

Future repository maintenance should include:

- Remove unused imports after confirming future integration plans.
- Ignore `\_\_pycache\_\_` directories in version control.
- Apply automated formatting tools (Black/Ruff) when appropriate.

These are maintainability improvements and are not Sprint 2 blockers.

Function Naming

`calculate\_profitability\_kpi()` currently computes profitability, leverage, efficiency and cash flow KPIs.

A Future refactor should rename the function to better reflects its responsibilities.

Sprint 2 reinforced several engineering principles.

- Never assume supplied financial datasets are correct.
- Validate business formulas independently of source data.
- Separate analytics logic from ETL orchestration.
- Document engineering decisions rather than silently applying workarounds.
- Build edge-case handling into the implementation rather than fixing failures after deployment.

Sprint 3 should focus on consuming the validated analytics layer rather than modifying it.

Recommendation priorities:

- 1. Screener Engine
- 2. Peer comparison
- 3. Health Intelligence

The Ratio Ending should now be treated as a dependency unless defects are discovered.

Sprint 2 successfully established the analytics foundation of the N100 Financial Ratio Engine has been implemented, validated, documented and tested. Major dataset inconsistencies have been investigated and engineering decision have been recorded.

The project is now ready o progress to Sprint 3.

- Area | Status
- Sprint Objective | Completed
- Ratio Engine | Completed
- Database Population | Completed
- Data Validation | Completed

- Edge Case Handling | Completed
- Automated Testing | 109/109
- Documentation | Completed
- Critical Issues | None

Sprint 3 retrospective

Source: [docs/Sprint3_retrospective.md](#)

N100 Financial Intelligence Platform

Sprint: Sprint 3 - Screener & Peer Comparison Engine (Days 15-21)

Status: Completed with two documented data limitations

Prepared On: Day 21

Deliver a fully functional financial screener with six preset filters and custom threshold support, and compute peer percentile rankings for all 11 peer groups across 10 metrics.

Three Sprint 2 exit criteria were still open when Sprint 3 began. They were closed before screener work started, because the growth presets depend on them.

- Item | Resolution
- CAGR columns absent from `financial\_ratios` | `revenue\_cagr\_5yr`, `pat\_cagr\_5yr`, `eps\_cagr\_5yr` and `composite\_quality\_score` added and populated
- `output/capital\_allocation.csv` never generated | Exported by `ratio\_engine.py`, 1,065 rows
- `src/analytics/cashflow\_kpis.py` missing | Cash flow KPIs moved out of `ratios.py`, re-exported for compatibility
- `src/etl/ratio\_edge\_case\_audit.py` empty | Implemented; regenerates the edge case log and reproduces the 18 ROE anomalies previously found by hand

The Sprint 2 retrospective recorded CAGR integration as deferred technical debt on the grounds that the schema had no CAGR fields. That reasoning was inverted: `src/etl/schema.sql` already declared all four columns, and only the live database file was missing them. The columns were added with `ALTER TABLE` and back-filled.

Screening universe

`src/screener/universe.py` assembles one row per company at its latest financial year, joining four tables. Two structural problems had to be solved first.

Duplicate company-year rows. Sprint 2 deliberately preserved source duplicates, so the Sprint 2 joins fan out: PNB carries 60 rows for 12 distinct years. Any screener built directly on `financial\_ratios` would count companies more than once. All analytics now de-duplicate on company-year before use.

Inconsistent period labels. Companies do not share a financial year end, and the raw files mix `Mar 2024`, `Dec 2023`, `TTM`, `Mar 2023 15` and `Mar 2016 9m`. `src/analytics/periods.py` centralises parsing so CAGR windows, latest-year selection and the `market\_cap` join all agree on what the latest year is.

Filter engine

`src/screener/engine.py` supports all 15 filterable metrics. Two business rules live in code rather than config:

1. **Financials D/E exemption.** An upper-bound D/E filter does not reject banks, NBFCs or insurers. The exemption deliberately does *not* apply to the equality test used by Debt-Free Blue Chip, because a bank with a D/E of 8.2 is not debt free. The sprint guide wording is "when D/E max

filter is applied", which supports the narrower reading.

- 2. **Debt Free interest coverage.** ``interest_coverage`` is null precisely when interest expense is zero. Such a company has infinite coverage and passes any ICR minimum. For every other metric, a missing value fails.

Composite quality score

Weighted 35% profitability, 30% cash quality, 20% growth, 15% leverage.

Each component is winsorised at P10/P90 before scaling to 0-100.

Winsorisation is doing real work here rather than being a formality. The

source data contains ROE values such as BEL at 4,744% and INDIGO at 893%, caused by equity capital and profit figures being on different scales.

Without capping, those two companies would compress every other company into a narrow band at the bottom of the profitability axis.

A missing metric scores a neutral 50 rather than 0. Scoring it 0 would punish a company twice for a documented edge case: a turnaround company already has a null PAT CAGR because its base year was a loss, and assigning it the worst possible growth score would compound that.

A sector-relative variant normalises within ``broad_sector``.

Peer percentiles

``src/analytics/peer.py`` computes SQL-style ``PERCENT_RANK``,

``(rank - 1) / (n - 1)``, on a 0-100 scale for 10 metrics across the 11 peer groups, writing 550 rows to the new ``peer_percentiles`` table.

Debt-to-equity is inverted so less debt earns the higher rank. Companies absent from every peer group return the message ``No peer group assigned`` rather than raising.

- Criterion | Result
- 6 presets each return 5-50 companies | **4 of 6** - see limitations
- ``peer_comparison.xlsx`` has 11 sheets | Pass - 11 sheets
- Peer percentile ranks correct | Pass - spot-checked on IT Services and FMCG
- All 14 DQ rule unit tests pass | Pass - 14/14
- Full test suite | Pass - 156/156

Preset results on the delivered dataset:

- Preset | Companies | Status
- Quality Compounder | 22 | Pass
- Value Pick | 2 | Below range
- Growth Accelerator | 18 | Pass
- Dividend Champion | 30 | Pass
- Debt-Free Blue Chip | 2 | Below range
- Turnaround Watch | 34 | Pass

Both preset shortfalls come from the supplied data, not the filter logic.

The specified thresholds are retained as the shipped defaults because the sprint guide is the governing specification and they are correct for live market data. The calibrated alternatives are documented in ``config/screener_config.yaml`` for the analyst to switch.

Value Pick returns 2 companies

``P/E`` and ``P/B`` come from the simulated ``market_cap`` dataset, in which the two are uncorrelated ($r = -0.11$). In real markets they move together, so a

company that is cheap on earnings is usually also cheap on book value.

15 companies pass `P/E < 20` and 9 pass `P/B < 3.0`, but only 2 pass both, which is roughly what independence predicts.

Relaxing to `P/E < 30` and `P/B < 6.0` returns 13 companies.

Debt-Free Blue Chip returns 2 companies

Only 3 of 92 companies report literally zero borrowings in their latest year. Using `D/E < 0.05`, the conventional market definition of debt free, returns 16 companies.

Source data defects confirmed

- Defect | Detail
- `companies.xlsx` truncated | 92 rows ending part way through the alphabet. ULTRACEMCO, UNIONBANK, UNITDSPR, VBL, VEDL, WIPRO, ZOMATO and ZYDUSLIFE appear in the statements with no master record
- SBIN missing from statements | SBIN is the designated Public Sector Banks benchmark but has no `financial_ratios` rows, so that peer group has no highlighted benchmark
- 119 duplicate company-year rows | Preserved per the Sprint 2 decision; de-duplicated in analytics
- ROE/ROCE snapshot mismatches | 18 companies, all categorised in `output/ratio_edge_cases.log`
- 1. `composite_quality_score` is latest-year only. The score is cross-sectional, so back-filling it onto historical rows would imply a comparison that was never made. Sprint 4 should decide whether it wants a historical score series.
- 2. `ROCE` is still not persisted. It is recomputed in the universe builder each run because the Sprint 2 schema has no `ROCE` column.
- 3. `make` is not installed on the development machine. The `Makefile` matches the documented command contract but was not executed; every target was verified by running its underlying module directly.
- 4. `book_value_per_share` remains unreconciled, carried over from Sprint 2.
- Verify a deferral before accepting it. The CAGR columns were recorded as blocked by the schema when the schema already defined them.
- Simulated data does not preserve the correlations that real screening thresholds assume. Presets that filter on two related valuation metrics behave very differently once that relationship is removed.
- A filter engine needs to distinguish "missing because the concept does not apply" from "missing because the data is absent". Debt Free interest coverage is the clearest case: the same null means infinite coverage, not unknown coverage.

The screener and peer layers should now be treated as dependencies.

Sprint 4 should consume `composite_quality_score`, `peer_percentiles` and the screener presets rather than modifying them, and should surface the SIMULATED labelling on every valuation metric it displays.

Sprint 4 retrospective

Source: docs/Sprint4_retrospective.md

N100 Financial Intelligence Platform

Sprint: Sprint 4 - Dashboard & Valuation Module (Days 22-28)

Status: Completed

Prepared On: Day 28

Deliver a working eight screen Streamlit dashboard on localhost:8501 that loads without errors for any of the 92 tickers, plus a valuation module producing `valuation\_summary.xlsx` with FCF yield, P/E flags and overvaluation labels.

Dashboard

- Screen | File | Contents
- Home | `01\_home.py` | 6 KPI tiles, sector donut, top 5 by composite score, sidebar year selector
- Company Profile | `02\_profile.py` | Search, company card, 6 KPI tiles, revenue/profit bars, ROE-ROCE dual axis, pros and cons
- Screener | `03\_screener.py` | 10 sliders, 6 preset buttons, live table, CSV download
- Peer Comparison | `04\_peers.py` | Group dropdown, Scatterpolar radar, benchmark-highlighted table
- Trend Analysis | `05\_trends.py` | Up to 3 overlaid metrics, YoY annotations
- Sector Analysis | `06\_sectors.py` | Revenue/ROE bubble chart sized by market cap, sector median bars
- Capital Allocation | `07\_capital.py` | Treemap over the 8 patterns with drill-down
- Annual Reports | `08\_reports.py` | BSE PDF links with unavailable badges

`src/dashboard/utils/db.py` wraps every query in

`@st.cache\_data(ttl=600)`. `src/dashboard/utils/ui.py` centralises

formatting so that a missing metric renders as `N/A` on every screen rather than crashing one of them.

The screener screen drives the same `ScreenerEngine` as the Sprint 3 batch export, so the dashboard and `output/screener\_output.xlsx` cannot disagree about what a filter means.

Valuation module

`src/analytics/valuation.py` computes FCF yield as free cash flow over market cap, each company's own five year median P/E, and the sector median P/E in the latest year. Flags follow the specification: Caution above 1.5x the sector median, Discount below 0.7x, Fair in between. Result across the 92 companies: 46 Fair, 30 Discount, 14 Caution, and 2 unflagged because they have no valuation data at all.

- Criterion | Result
- All 8 screens load without errors for any of the 92 tickers | Pass
- Company Profile loads in under 3 seconds | Pass - p95 0.16s, worst case 0.99s
- Screener CSV download produces valid headers | Pass
- `valuation\_summary.xlsx` has 92 rows with all required columns | Pass
- Test suite | Pass - 183 tests, up from 156

The 92-ticker sweep ran every company through the Profile, Trends and Reports screens: 270 page renders, zero exceptions after the fix below.

PNB crashed the Company Profile screen.

`calculate\_roce` raised `TypeError: unsupported operand type(s) for +: 'NoneType' and 'int'`. PNB reports no operating profit in any of its 61 rows, which is normal for a bank, and the profile screen guarded only

equity and borrowings before calling the function.

The fix went into `calculate_roce` itself rather than the screen. Three call sites now depend on that function, and guarding each caller separately would have left the same trap for the next one. The function returns `None` when any input is missing, which is what the rest of the pipeline already expects.

This is exactly the class of bug the Day 27 sweep exists to catch: it only appears on 1 of 92 companies and would have surfaced during the demo.

Annual report link checking is opt-in

The sprint guide asks for a "Report unavailable" badge when a URL returns 404. Verifying every link on each render would fire dozens of requests to bseindia.com behind a single page load and break the three second target. The screen renders instantly from the database, flags missing URLs immediately, and only contacts BSE when the analyst ticks the verification box. Results are cached for an hour.

Preset buttons approximate three presets

The guide specifies ten sliders, and three preset rules cannot be expressed as one:

- Preset | Rule that has no slider | Dashboard | Excel
- Debt-Free Blue Chip | `D/E == 0` exactly, revenue > 5,000 Cr | 33 | 2
- Dividend Champion | payout ratio < 80% | 33 | 30
- Turnaround Watch | D/E falling year on year | 40 | 34

The screen documents this in an expander pointing at `make_screener` for the full rules. The alternative, adding sliders the guide did not ask for, would have diverged from the specification.

Composite score is computed in process

`get_universe()` builds the screening universe and scores it inside the dashboard rather than reading a scored table. The composite score is cross-sectional, so it has to be recomputed whenever the universe changes. The ten minute cache makes this a one-off cost per session.

- Finding | Detail
- Only 10 broad sectors exist | The project document and Sprint 4 guide both refer to 11 sectors, but `sectors` contains 10. The donut renders the 10 actual sectors plus an Unclassified bucket for the 2 orphan tickers
- `prosandcons` covers 16 of 92 companies | The profile screen shows an explanatory note rather than an empty panel for the other 76
- 52 annual report URLs are missing | Those years render the red unavailable badge
- 3 companies have under 10 years of history | ADANIGREEN, JIOFIN and LIC. Charts show the available history with a caption
- 2 tickers never appear in the selector | ULTRACEMCO and UNIONBANK have no company master record, so they cannot be searched for. They still appear in aggregate metrics
- One capital allocation sign combination is unnamed | The guide lists eight patterns, but two of them (Reinvestor and Shareholder Returns) share the `(+,-,-)` signs, so only seven of the eight possible combinations are named. ICICIPRULI and JIOFIN are `(-,+,-)` and render as Unknown Pattern, which the screen explains
- Average ROE reads 122.63% | Skewed by the same source defect behind the Sprint 2 ROE anomalies: a few companies report equity that is not on the same scale as their profit. The tile carries a tooltip giving the median (15.8%) and pointing at the edge case log
- 1. `make` is still not installed on the development machine. Targets

were verified by running the underlying modules directly.

- 2. ****The valuation module reads simulated multiples.**** Every flag is a demonstration of the logic, not an investment view. The workbook carries a Notes sheet saying so.

- 3. ****The 92-ticker sweep is a manual QA script****, not part of `make test`. The suite covers the eight screens plus three regression tickers to keep the default run fast.

- 4. ****Sector-relative composite score is computed but not surfaced**** on any screen yet.

- Sweeping every ticker through every screen found a defect that no amount of spot-checking would have. One company in 92 broke a screen, and it was a bank, which is the segment most likely to be demoed.

- Fix a missing-data crash in the shared function, not in the caller that happened to hit it first.

- When a specification limits the UI, say so in the UI. The screener presets genuinely cannot reproduce the batch export, and a one paragraph expander is cheaper than an analyst filing a bug.

Sprint 5 covers cash flow intelligence, NLP over the pros and cons and analysis tables, company tearsheets and sector PDFs. Two constraints carry forward: `prosandcons` covers only 16 companies and `analysis` only 20, so any NLP deliverable should state its coverage up front rather than implying it spans the index.

Sprint 5 retrospective

Source: docs/Sprint5_retrospective.md

N100 Financial Intelligence Platform

Sprint: Sprint 5 - Intelligence, NLP & PDF Reports (Days 29-35)

Status: Completed

Prepared On: Day 35

Auto-generate pros and cons for all 92 companies with confidence scores, classify every company by CFO quality, CapEx intensity and capital allocation pattern, and generate 92 company tearsheet PDFs plus 11 sector PDFs with no text overflow or layout errors.

- Deliverable | Output | Result
- Analysis text parser | `output/analysis\_parsed.csv` | 63 values parsed, 17 logged as failures
- Parse failure log | `output/parse\_failures.csv` | 17 rows, all TTM or Last Year labels
- Pros and cons generator | `output/pros\_cons\_generated.csv` | 519 statements, 381 pros and 138 cons
- Cash flow intelligence | `output/cashflow\_intelligence.xlsx` | 92 rows, 3 sheets
- Distress alerts | `output/distress\_alerts.csv` | 13 companies
- Pattern changes | `output/pattern\_changes.csv` | 530 year-over-year moves
- Company tearsheets | `reports/tearsheets/` | 91 PDFs, 85 to 106 KB
- Skipped tearsheets | `output/skipped\_tearsheets.csv` | 1 company
- Sector reports | `reports/sector/` | 11 PDFs
- Portfolio summary | `reports/portfolio/portfolio\_summary.pdf` | 93 pages, 145 KB
- Criterion | Result
- At least 1 pro and 1 con for every company | Pass - 0 gaps
- All tearsheets exist and are at least 30 KB | Pass - 91 of 91, smallest 85 KB
- No text overflow and no blank pages | Pass - all 91 are exactly 2 pages
- `cashflow\_intelligence.xlsx` has 92 rows with all required columns | Pass
- Test suite | Pass - 229 tests, up from 183

The guide allows the tearsheet count to be 92 minus skipped tickers.

JIOFIN was skipped because it carries 2 years of data against the 3 year floor, so 91 is the expected total.

Page count is used as the overflow check. ReportLab flows content that does not fit onto an extra page, so a tearsheet that is exactly 2 pages cannot have overflowed its frames. All 91 are exactly 2 pages.

The Day 29 cross-validation turned out to be the most useful thing in the sprint. The analysis dataset carries the vendor's own published 5 year growth figures as text, which gave a way to check the Sprint 2 CAGR engine against a source it had never seen.

- Company | Metric | Vendor | Computed | Divergence
- HDFCBANK | Sales growth | 22.0% | 21.95% | 0.05
- HDFCBANK | Profit growth | 23.0% | 23.87% | 0.87
- INFY | Sales growth | 13.0% | 13.20% | 0.20
- INFY | Profit growth | 11.0% | 11.24% | 0.24
- SBILIFE | Sales growth | 24.0% | 24.23% | 0.23
- SBILIFE | Profit growth | 6.0% | 7.37% | 1.37
- TCS | Sales growth | 10.0% | 10.46% | 0.46
- TCS | Profit growth | 8.0% | 7.87% | 0.13

All 8 comparisons agree within 1.4 percentage points against a 5 point tolerance. The vendor rounds to whole numbers, which accounts for most of the remaining gap. This is independent confirmation that the CAGR engine

and the period-ordering logic are both correct.

The exit criterion needed a relative fallback

Implementing the 24 specified rules exactly produced a gap: 47 companies tripped no con rule and 4 tripped no pro rule, against an exit criterion requiring at least one of each for every company.

The gap was not a bug. TCS, ITC and NESTLEIND all carry negative net debt, interest coverage above 40x, rising revenue and no loss, so none of the 12 con rules apply to them. BHEL sits at the other end with 1.2% ROE, 3% operating margin and revenue shrinking at 4.7% a year, so none of the 12 pro rules apply.

Two extra rules were added rather than loosening the specified 24:

- `PRO-13` and `CON-13` name the company's strongest and weakest metric relative to its sector.

- They fire only when no absolute rule did.
- Their confidence is capped at 78, below the absolute rules, and the text says "no absolute red flag, but ranks in only the Nth percentile", so a relative weakness can never be read as a red flag.

The two tickers with no sector are ranked against the whole index instead, since a group of two carries no ranking information.

The distress flag needs a sector caveat

`CFO < 0 AND CFF > 0` flagged 13 companies, of which 9 are profitable

Financials: AXISBANK, BAJAJFINSV, BAJFINANCE, BANKBARODA, CHOLAFIN, PFC, PNB, RECLTD and SHRIRAMFIN. BAJFINANCE was flagged as distressed on 14,451 Cr of net profit.

For a lender this is the ordinary signature of growth: expanding the loan book is an operating outflow and taking deposits or borrowing is a financing inflow. The flag itself is left exactly as specified, and `distress\_alerts.csv` carries a `structurally\_normal\_for\_sector` column plus an `interpretation` column so a profitable lender is never read as distressed. GRASIM and M&M are also flagged and both run large financing arms, so the same mechanism is at work, but they are left unmarked because they are not classified as Financials.

This mirrors the carve-out the sprint guide itself applies to the D/E screener filter.

Arrows are drawn, not typed

The portfolio summary needs up, down and flat arrows. The standard PDF fonts have no arrow glyphs, and substituting one silently renders as a blank box in some viewers, so the arrows are ReportLab polygons.

Charts are matplotlib images

Tearsheet charts are rendered by matplotlib into an in-memory PNG and placed as images rather than redrawn in ReportLab primitives. That keeps one charting approach across the project instead of two.

- Finding | Detail
- The analysis table covers 5 companies | 20 rows for HDFCBANK, INFY, SBILIFE, TCS and one other, not 92. Every parser output is a 5 company sample and says so on each run

- 17 of 80 text values do not parse | All are `TTM:` or `Last Year:` labels. The specified regex requires a digit before "Year", so these are correct rejections rather than failures
- `prosandcons` was not usable as a source | It covers 16 companies, so all 519 statements are generated from financial rules rather than the supplied text
- CON-09 never fires | No company shows EPS declining across 3 strictly consecutive years. The rule is correct and simply does not apply to this dataset
- The cashflow table holds 100 tickers | 8 more than the platform universe. Without a filter the intelligence report ran to 100 rows with 8 unclassified, so it is now restricted to the 92 companies that survived the Sprint 2 statement joins
- Operating profit is used as EBITDA | Correct, but the original justification was wrong. It was recorded as "the datasets carry no depreciation line"; in fact `profitandloss.depreciation` is fully populated across all 1,276 rows. The right reason is that the source P&L reports operating profit *before* depreciation, so it is EBITDA by construction. Verified against the identity `operating profit + other income – interest – depreciation = profit before tax`, which holds with a median residual of 1.0 across 1,263 rows against 1,317 for the post-depreciation reading. Corrected on Day 45
- 11 sector PDFs, 10 real sectors | The eleventh is the Unclassified bucket holding the two tickers with no company master record
- 1. **No PDF renderer is installed**, so overflow was verified by page count rather than by rendering each page to an image. The four chart types were inspected individually.
- 2. **`make` is still not installed.** Targets were verified by running each module directly.
- 3. **The tearsheet batch takes several minutes** for 91 companies because each one renders four matplotlib figures. Acceptable as a batch job, but it is not something to run inside a web request.
- 4. **Sector reports carry no charts**, only median tiles and a company table. The guide asks for medians and a company list, which is met, but a chart per sector would read better.
- A dataset that duplicates work already done is worth more as a check than as a source. The analysis table only covers 5 companies, but those 5 confirmed the CAGR engine against an independent figure.
- An exit criterion can conflict with the rules that feed it. Saying so and adding a labelled fallback is better than quietly loosening the rules until the number comes out right.
- A rule that is correct in general can be wrong for a sector. Negative operating cash flow means trouble for a manufacturer and growth for a lender, and the output has to carry that distinction or it will be misread.

Sprint 6 covers clustering, the FastAPI server with 16 endpoints, the pytest HTML report and final documentation. The API should read the existing outputs rather than recompute them; the composite score and the tearsheet batch are both too slow to sit inside a request. The cached loader in `src/dashboard/utlis/db.py` is the pattern to follow.

Sprint 6 retrospective

Source: [docs/Sprint6_retrospective.md](#)

N100 Financial Intelligence Platform

Sprint: Sprint 6 - API Server, Clustering & Final QA (Days 36-45)

Status: Completed. 20 of 20 acceptance gates passing, 23 of 23 deliverables present.

Prepared On: Day 45

Get all 16 FastAPI endpoints live and returning correct data, assign all 92 companies to one of 5 labelled archetypes, show 60+ tests with zero failures, and verify all 20 acceptance gates for project sign-off.

- Measure | Result
- Acceptance gates | **20 of 20 PASS**
- Deliverables | **23 of 23 present**
- Test suite | **298 passing**, 0 failures
- API endpoints | 20 live (16 required plus 4 convenience)
- Companies clustered | 92 of 92 into 5 archetypes
- Deliverable | Output
- KMeans clustering | ``output/cluster_labels.csv``, ``output/cluster_profiles.csv``
- Elbow curve | ``reports/elbow_plot.png``
- Correlation heatmap | ``reports/correlation_heatmap.png``
- Outlier detection | ``output/outlier_report.csv``
- Portfolio percentiles | ``output/portfolio_stats.csv``
- FastAPI server | ``src/api/`` with 7 routers
- OpenAPI 3 spec | ``docs/openapi.json``
- Test report | ``reports/pytest_report.html``
- SQLite indexes | ``src/etl/create_indexes.py``, 12 indexes
- Performance notes | ``output/perf_notes.md``
- Analyst guide | ``docs/analyst_guide.pdf``, 11 pages
- Acceptance checklist | ``docs/acceptance_checklist.pdf``
- Handover archive | ``output/final_deliverables/``, 214 files

Clustering needed winsorisation to produce anything useful

The specified pipeline is impute, StandardScaler, KMeans with k=5. Run exactly that way, the result was unusable:

- Cluster | Companies | Mean ROE | What it actually was
- 4 | 2 | 4,280% | BEL and HAL, the ROE data artifacts
- 2 | 1 | 15.6% | CIPLA alone, on a 229% FCF CAGR
- 0 | 58 | 21.6% | 63% of the index dumped together

Three of the five clusters were spent isolating outliers. StandardScaler normalises the spread but does nothing about the fact that BEL's 4,744% ROE is a data artifact, so that single company defined an entire axis.

Capping every feature at its 5th and 95th percentile before scaling fixed it. The clusters became 13 / 26 / 28 / 15 / 10 and each one has a recognisable financial character. This mirrors the P10/P90 winsorisation the composite score has used since Sprint 3, and for the same reason.

The example cluster names did not fit the data

The guide offers five example names. Assigning them mechanically produced two clear errors: the cluster labelled "Emerging Growth" had a mean D/E of 6.8 and was entirely banks, and the one labelled "Distressed

or Turnaround" carried 81% operating margins.

The naming logic now derives each label from the cluster's most distinctive feature:

- Cluster | Companies | Name | What separates it
- 4 | 10 | High-Quality Compounders | 60% mean ROE, low leverage
- 0 | 13 | High-Margin Franchises | 81% operating margin
- 1 | 26 | Emerging Growth | 15% revenue and 19% FCF growth
- 2 | 28 | Defensive Dividend Payers | Mature, 7.6% growth
- 3 | 15 | Leveraged Financials | Mean D/E of 6.8, banks and NBFCs

Two names differ from the guide's examples. The guide describes them as examples and asks for names based on which companies are actually in each cluster, so this follows the instruction rather than departing from it.

No cluster in this universe is genuinely distressed: the weakest still averages 13% ROE.

The API caches rather than recomputes

The composite quality score is cross-sectional, so it cannot be computed for one company on request. Building it per call would put a full index recomputation behind every endpoint. It is built once per process with ``lru_cache`` in ``src/api/dependencies.py``. A warm ``/screener`` call takes 13 ms; a cold one takes about a second.

Pearson correlation is reported with a caveat

The heatmap is Pearson as specified, but on this data Pearson overstates several relationships. ROE against asset turnover reads +0.96 on Pearson and +0.49 on Spearman; net profit margin against interest coverage reads +0.79 against +0.09. Those gaps mean the coefficient comes from a few companies at the extreme of both distributions, not from a relationship that holds across the index. The figure now carries a footnote naming the two worst offenders and their rank-correlation values.

Gate AC-05 uses the vendor's own figures as the manual check

The gate asks for a Revenue CAGR spot-check against a manual Excel calculation. The Sprint 5 parser already extracts the vendor's published 5-year growth figures from the ``analysis`` table, which is a stronger check than recomputing our own formula by hand: it is an independent source. Worst divergence across the available companies is 0.46 percentage points, and the vendor rounds to whole numbers.

black was deliberately not run

Day 44 asks for a final pass with ``black src/ tests/`` and ``ruff check src/ tests/``. Ruff passes clean across the whole codebase.

Black was not run.

Black would reformat 72 of the 92 files. Not because of anything wrong with them, but because the project has used 3-space indentation since Sprint 1 and black enforces 4-space with no option to configure it. The result would be a purely cosmetic diff touching almost every file in the repository, applied on the last day, obscuring the real history of what changed and when.

The deviation is recorded here rather than quietly skipped. Reversing the decision is one command:

```
python -m black src/ tests/
```

The handover archive is not committed

``output/final_deliverables/`` is a byte-for-byte copy of 19.1 MB of files already tracked elsewhere in the repository. It is gitignored and regenerates with one command. Committing it would double the repository size to hold nothing new.

Both were found while working on ``gitignore`` in the final pass, and neither was introduced this sprint.

``__init__.py`` was gitignored

``gitignore`` contained a bare ``__init__.py`` entry. Ten of the twenty package markers on disk were untracked as a result, including ``src/api/__init__.py``, ``src/nlp/__init__.py`` and four test package markers. The other ten predated the entry and were already tracked, so the repository was in a half-working state that a fresh clone would have exposed. The entry has been removed.

``env`` and ``vscode`` had merged into one pattern

The two lines had run together as ``env.vscode``, a single pattern matching neither. ``env`` was therefore not ignored. No ``env`` file currently exists, so nothing leaked, but a future one would have been committed. Both are now on separate lines.

All 20 gates, with the evidence recorded at run time:

- Gate | Criterion | Evidence
- AC-01 | companies = 92 | 92
- AC-02 | 90%+ have 10 years of statements | 84/92 = 91.3%
- AC-03 | foreign_key_check clean | 0 violations
- AC-04 | financial_ratios >= 1,100 | 1,184 rows
- AC-05 | Revenue CAGR spot-check | worst divergence 0.46 pts
- AC-06 | ROE within 5% for 5+ companies | 70 companies
- AC-07 | Quality preset returns 10-50 | 22 companies
- AC-08 | Profile screen under 3s | p95 0.16s, worst 0.99s
- AC-09 | Screener CSV well-formed | 24 columns
- AC-10 | No overflow in 5 tearsheets | all exactly 2 pages
- AC-11 | /health returns 200 | HTTP 200
- AC-12 | TCS ratios 10+ years | 12 years
- AC-13 | API matches screener_output.xlsx | 22 vs 22, identical
- AC-14 | peer_percentiles has 11 groups | 11
- AC-15 | 92 companies have a cluster_id | 92, 5 clusters
- AC-16 | 92 companies have a pro and a con | 0 missing
- AC-17 | Tearsheets exist, 30 KB+ | 91 PDFs + 1 skipped, 0 undersized
- AC-18 | 60+ tests, 0 failures | 298 collected
- AC-19 | validation_failures.csv columns | 188 rows, all columns
- AC-20 | analyst_guide.pdf 10+ pages | 11 pages

Regenerate with ``make acceptance``.

1. ``make`` is not installed on the development machine. Every target was verified by running its module directly. The Makefile itself was checked for tab integrity and literal-escape errors but never

executed.

- 2. ****No PDF renderer is available****, so tearsheet overflow is verified by page count rather than by rendering pages to images. The four chart types were inspected individually.
- 3. ****The API is read-only and unauthenticated.**** Correct for internal use, and CORS is deliberately open. It must not be exposed publicly in this form.
- 4. ****The tearsheet batch takes minutes.**** It is an offline job and must stay one.
- 5. ****`prosandcons` and `analysis` cover 16 and 5 companies.**** Any future NLP work should state its coverage rather than implying index-wide data.
- Scaling is not the same as robustness. StandardScaler put every feature on a comparable spread and still let one company's bad data define a cluster. Capping the tails was what actually made the clustering mean something.
- When a specification offers example labels, check them against the result before adopting them. Two of the five given names described something the data does not contain.
- Report the statistic you were asked for, and report where it misleads. Pearson was specified; the Spearman comparison is what tells a reader whether to trust a given coefficient.
- Repository hygiene deserves a deliberate pass. A gitignored `\_\_init\_\_.py` and two merged patterns had been quietly wrong for sprints, and neither would have shown up in a test run.

All six sprints are complete. The platform loads twelve datasets, computes 30+ KPIs across 1,184 company-year rows, screens on 15 metrics with 6 presets, ranks 11 peer groups, serves an eight-screen dashboard and a 16-endpoint API, and produces 91 company tearsheets, 11 sector reports, a 93-page portfolio summary and 92 radar charts.

Every source data defect found along the way is documented rather than worked around silently: the truncated company master, the simulated valuation dataset, the ROE scale artifacts, the duplicate company-year rows and the coverage limits of the analysis and pros-and-cons tables. A reader of the retrospectives can tell what the platform knows from what it is guessing.